

Entwicklerdoku – OKF-Wissensdatenbank pflegen (xlsxBuilder)

Generiert aus der OKF-Wissensdatenbank von xlsxBuilder —
docs/okf/xlsxbuilder/conventions/okf-entwicklerdoku.md

Frontmatter

type: Developer Guide

title: Entwicklerdoku – OKF-Wissensdatenbank pflegen

description: Wie das OKF-Bundle dieses Projekts aufgebaut ist, wie man neue Konzepte anlegt, was automatisch passiert und was bewusst Handarbeit bleibt.

resource: tools/kb/generate_okf_index.py

tags: [developer-guide, okf, graphify, process]

timestamp: '2026-07-06T09:00:00+02:00'

Überblick

OKF (Open Knowledge Format) ist ein offenes, vendor-neutrales Format für Wissen als Markdown-Dateien mit YAML-Frontmatter — spezifiziert in

[GoogleCloudPlatform/knowledge-catalog/okf/SPEC.md](https://googlecloudplatform.github.io/knowledge-catalog/okf/SPEC.md).

Kein Schema-Registry, keine zentrale Autorität, keine zwingende Tooling-Abhängigkeit — wer `cat` und `git clone` kann, kann OKF lesen und versionieren.

Abgrenzung zu graphify: Der Wissensgraph unter [graphify-out/](#) (siehe

[GRAPH_REPORT.md](#)) wird **deterministisch aus dem Java-AST** erzeugt — er kennt jede

Klasse/Methode, aber keine Semantik. Das OKF-Bundle unter [docs/okf/xlsxbuilder/](#) ist die **kuratierte, von Menschen/LLM geschriebene Prosa** darüber: Architektur-Entscheidungen, Entwurfsmuster, Sicherheitshinweise, Warum-Fragen. Beide ergänzen sich (Hybrid-Strategie, siehe Abschnitt „Automatisierung“).

Bundle-Struktur dieses Projekts

```
docs/okf/xlsxbuilder/
  ■■■ index.md          # GENERIERT – nie von Hand editieren
  ■■■ log.md            # Änderungshistorie, ISO-Datum, neueste zuerst
  ■■■ architecture/    # Architektur-Entscheidungen, Entwurfsmuster
  ■   ■■■ index.md      # GENERIERT
  ■   ■■■ components/   # Öffentliche/wichtige Klassen und Komponenten
  ■   ■■■ index.md      # GENERIERT
  ■■■ conventions/     # Build/Release/Code-Stil/Prozess (auch diese Datei)
  ■■■ index.md         # GENERIERT
```

Jede `.md`-Datei außer `index.md/log.md` ist ein **Concept** (eigenständiges

Wissensdokument). `index.md`-Dateien fassen den Inhalt eines Verzeichnisses zusammen

(Progressive Disclosure) und werden von `tools/kb/generate_okf_index.py` erzeugt —

Handänderungen daran gehen beim nächsten Commit verloren.

Frontmatter-Konvention

```
---
type: <Kind>                                # PFLICHT — z. B. "Architecture Concept",
                                             #   "Library Component"/"Vaadin Component",
                                             #   "Convention", "Developer Guide"

title: <Anzeigename>
description: <Ein Satz Zusammenfassung>
resource: <Pfad zur zugehörigen Datei, repo-relativ>
tags: [<Stichwort>, ...]
timestamp: '<ISO 8601>'
---
```

- **Querverweise** immer bundle-root-absolut: `XmlViewer` (beginnt mit `/`, relativ zur Bundle-Wurzel `docs/okf/xlsxbuilder/`) — **nicht** `../components/...` (der Index-Generator warnt bei relativen `../`-Links).
- **Konventionelle Überschriften** (optional, aber empfohlen wo zutreffend): `# Schema` (Methoden-/Klassentabelle), `# Examples` (Codebeispiele), `# Citations` (Quellenangaben, z. B. Links ins README).

Neues Konzept anlegen

1. Passenden Unterordner wählen (`architecture/` für Entwurfsentscheidungen, `components/` für konkrete Klassen/Komponenten, `conventions/` für Build/Release/Prozess).
2. Datei anlegen mit vollständigem Frontmatter (siehe oben) — **type ist Pflicht**, ohne das meldet der Index-Generator eine Konformitätswarnung.
3. Body schreiben: kurzer Überblick, dann `# Schema/# Examples/# Citations` wo passend. Querverweise zu verwandten Konzepten bundle-root-absolut setzen.
4. **Nichts an `index.md` ändern** — das übernimmt der Pre-Commit-Hook automatisch.
5. `docs/okf/xlsxbuilder/log.md` einen Eintrag unter dem heutigen ISO-Datum hinzufügen (neueste zuerst, siehe vorhandene Einträge als Vorlage).

Automatisierung

Der versionierte Hook `.githubhooks/pre-commit` läuft bei jedem Commit und macht **automatisch, ohne LLM** (deterministisch, kein API-Key nötig):

- **.java-Dateien im Commit** → `graphify update .` (AST-Rebuild) → `graphify-out/graph.json/GRAPH_REPORT.md/graph.html` werden neu erzeugt und mit ins Commit aufgenommen.
- **docs/okf/-Dateien im Commit** → `tools/kb/generate_okf_index.py` → alle betroffenen `index.md`-Dateien werden neu geschrieben und mitcommittet.

Der Hook blockiert nie einen Commit — fehlt `graphify` oder Python, erscheint nur eine Warnung, der Commit läuft trotzdem durch.

Was der Hook bewusst NICHT macht: die eigentliche Konzept-**Prosa**

(Architekturbeschreibung, Sicherheitshinweise, Beispiele) schreiben. Das ist und bleibt Handarbeit — durch euch oder durch eine Claude-Code-Session (siehe Pre-Commit-Routine unten). Das ist die bewusste **Hybrid-Strategie**: deterministisch wo möglich, kuratiert wo Semantik gefragt ist.

Graphify-Zusammenspiel

- **Graph-First-Regel:** Bei Fragen zur Codebasis („wo ist X?“, „was nutzt Y?“) zuerst den Wissensgraphen befragen statt frei zu grepen:

```
graphify query "Wie hängen X und Y zusammen?"
```
- **Vor einem Commit zur Kontrolle:** `graphify update` . manuell laufen lassen (der Hook macht es beim Commit ohnehin, aber so seht ihr das Ergebnis vorher).
- Der aktuelle Graph lässt sich auch direkt im Browser ansehen: graphify-out/graph.html.

Pre-Commit-Routine

Die verbindliche Routine steht in `AGENTS.md` unter „Knowledge Base (graphify + OKF)“ — dort ist sie die Single Source of Truth, hier nur der Verweis, um Redundanz zu vermeiden. (`CLAUDE.md` importiert `AGENTS.md` per `@AGENTS.md` — Claude Code liest die Routine also automatisch mit.) Kurzfassung: **vor jedem einzelnen Commit** mit Code-/Architekturänderungen (nicht erst am Sitzungsende) berührte Konzepte aktualisieren → `log.md`-Eintrag → `graphify update` . zur Kontrolle → erst dann committen, Doku und Code im selben Commit.

Bekannte Stolpersteine

- **Windows-Store-Python-Stub:** `command -v python3` findet unter Windows manchmal nur den WindowsApps-Alias-Stub (kein echtes Python). Der Hook validiert deshalb per Probelauf (`python3 -c "import sys"`), nicht nur per `command -v`.
- **Zeilenenden:** `.github/hooks/pre-commit` muss mit **LF** committet sein (siehe `.gitattributes`), sonst scheitert `sh` unter Git für Windows.
- **Reservierte Dateinamen:** `index.md` und `log.md` dürfen nie als normale Konzeptdateien verwendet werden.
- **graphify-out/-Ausschlüsse im .gitignore:** `.graphify_*`, `cache/`, `manifest.json`, `cost.json` und datumsbenannte Snapshot-Ordner (`graphify-out/JJJJ-MM-TT/`) sind maschinenlokal (absolute Pfade, mtimes) und gehören **nicht** ins Repo — nur `graph.json`/`GRAPH_REPORT.md`/`graph.html` werden versioniert.

Verifikation vor dem Commit

- `python tools/kb/generate_okf_index.py` **zweimal** laufen lassen — der zweite Lauf darf keine Änderung mehr melden (Determinismus-Check).
- Warnungen des Generators (`[kb] WARNUNG (OKF): ...`) ernst nehmen — meist fehlendes `type:-`Feld oder ein relativer `../`-Link.

Citations

[1] [Open Knowledge Format v0.1 – Spezifikation](#)